

Mini E-commerce Distribuído

Relatório Técnico ? Arquitetura de Microsserviços

Disciplina: Fundamentos de Computação em Nuvem | Individual | Valor: 3,0 pontos

Q1

Como a comunicação entre os microsserviços foi implementada?

A comunicação entre microsserviços foi implementada via HTTP/REST, com o API Gateway atuando como ponto de entrada único para todos os clientes externos. O Gateway centraliza o roteamento e repassa as requisições aos serviços usando httpx assíncrono.

Fluxo principal: Cliente ? Gateway (:8000) ? Usuários (:5001) / Produtos (:5002 e :5012) / Pedidos (:5003). Todos os contratos de API foram definidos com FastAPI e Pydantic, gerando documentação Swagger automática em /docs de cada serviço. Payloads em JSON.

Existe também comunicação serviço-a-serviço: o serviço de Pedidos chama o Gateway para atualizar estoque ao criar ou cancelar um pedido, usando o header X-Service-Secret para autenticação interna. Nenhum serviço compartilha código ou memória com outro ? cada um roda em processo e container independente.

Q2

Qual estratégia de consistência foi adotada na replicação? Por quê?

Foi adotada consistência FORTE (strong consistency) para as réplicas do serviço de Produtos.

O Gateway implementa um protocolo síncrono de escrita: toda operação de POST /products ou PATCH /products/{id}/stock é propagada simultaneamente para as duas réplicas via asyncio.gather(). O sucesso só é confirmado ao cliente quando ambas retornam HTTP 200/201. Se qualquer réplica falhar, a outra é revertida via rollback automático e um erro 500 é retornado.

Consequência: se uma réplica fica offline, escritas são bloqueadas com 503 mas leituras continuam via round-robin na réplica saudável. As réplicas nunca divergem pois nenhuma escrita é aceita enquanto uma delas está fora.

Justificativa: em e-commerce, inconsistência de estoque causaria overselling ? vender produtos sem estoque disponível. A consistência forte foi preferida em detrimento da disponibilidade total de escritas, pois o custo de um produto vendido sem estoque supera o de uma indisponibilidade temporária de escrita.

Q3

O que acontece se o Serviço de Pedidos cair? O restante continua?

Sim ? o restante do sistema continua funcionando normalmente com degradação parcial isolada ao serviço de Pedidos:

- Registro, login e consulta de usuários: funcionamento normal
- Listagem, busca e criação de produtos: funcionamento normal
- Criar ou visualizar pedidos: retorna 503 Service Unavailable
- Dashboard de monitoramento (/health): exibe Orders como OFFLINE

O Gateway detecta a falha via heartbeat: envia GET /health ao serviço de Pedidos a cada 5 segundos com 2 tentativas por ciclo. Ao detectar falha, registra no log: "Service 'orders' DOWN at 2026-06-14T..." e passa a retornar 503 para qualquer requisição de pedidos.

Ao reiniciar, o heartbeat do próximo ciclo detecta a recuperação e registra "Service 'orders' RECOVERED". O roteamento é retomado automaticamente sem necessidade de sincronização ? o banco de dados persiste em volume Docker.

Q4

Como o JWT garante que usuário comum não crie produtos?

O mecanismo funciona em duas camadas: assinatura criptográfica e verificação de papel (role), que juntas impedem qualquer tentativa de escalada de privilégios.

Camada 1 ? Emissão do token:

Ao fazer login, o serviço de Usuários gera um JWT assinado com HS256 e a chave secreta JWT_SECRET. O payload contém userId, email, role:"user" e exp (expiração em 24h). Um usuário comum sempre recebe role:"user". Qualquer adulteração no payload invalida a assinatura HMAC e o endpoint retorna 401 Unauthorized.

Camada 2 ? Validação no endpoint:

O endpoint POST /products usa a dependência verify_admin() que decodifica o JWT com a mesma JWT_SECRET e verifica payload["role"] == "admin". Se o role for "user", retorna 403 Forbidden antes de qualquer processamento.

O API Gateway repassa o header Authorization: Bearer <token> intacto para os serviços internos. Mesmo que um cliente tente contornar o gateway e chamar o serviço de Produtos diretamente, a validação JWT ocorre no próprio serviço ? não apenas no gateway.

Q5

Quais limitações em relação a um sistema real de produção?

- Gateway sem redundância (SPOF): sem instâncias paralelas; em produção seria executado múltiplas vezes atrás de um load balancer (AWS ALB, Nginx Plus).
- SQLite não escalável: não suporta múltiplos processos com segurança; produção usaria PostgreSQL com connection pooling (asyncpg).
- Sem circuit breaker: falhas em cascata não são isoladas; produção usaria o padrão Circuit Breaker (biblioteca pybreaker).
- JWT sem revogação: token comprometido permanece válido até expirar (24h); produção usaria blocklist de tokens em Redis ou tokens de curta duração com refresh.
- Sem rate limiting: sem proteção contra brute force no login nem abuso de endpoints; produção adicionaria rate limiting por IP no gateway.
- Heartbeat booleano: detecta apenas liveness, não latência nem throughput; produção usaria Prometheus + Grafana com métricas detalhadas por endpoint.
- Rollback não-atômico: falha durante o rollback de replicação pode deixar réplicas divergentes; produção usaria 2-phase commit ou event sourcing.
- Certificados auto-assinados: adequados apenas para desenvolvimento; produção utilizaria CA confiável (Let's Encrypt ou AWS Certificate Manager).